

# Test case prioritization for changed code using nature inspired optimizer

Babita Pathik<sup>a,\*</sup>, Nikhlesh Pathik<sup>b</sup> and Meena Sharma<sup>c</sup>

<sup>a</sup>*Department of CSE, TIT, Bhopal, India*

<sup>b</sup>*Department of CSE, ASET, Amity University, Gwalior, India*

<sup>c</sup>*Department of Computer Engineering, IET, DAVV, Indore, India*

**Abstract.** The software development and maintenance phase succeeded with significant regression testing activity. The software must be re-tested every time it upgrades to preserve its quality. Software testing as a whole is an expensive and tedious task due to resource constraints. Using the prioritization technique implies regression testing to re-test software after it has been modified. In this situation, the prioritization technique can use information acquired about earlier test case executions to generate test case orderings. The approaches for test case prioritization arrange them all in such a sequence that maximizes their efficacy in accomplishing specific goals. This paper presents a hybrid technique for change-testing or regression testing through test case prioritization. The suggested method first generates the test cases, then clustered in untested and unimportant groups using kernel-based fuzzy c-means clustering technique. The appropriate test cases are then considered for prioritization using the grey wolf optimizer. The results compared with the approaches such as ant colony, particle swarm, and genetic algorithm optimization method, and it is observed that the proposed approach efficiency increased by 91% of fault detection rate.

**Keywords:** Clustering, fuzzy c-means, grey wolf optimizer, test case prioritization, test suite augmentation

## 1. Introduction

Testing is a time-consuming and expensive process. It accounts for approximately half of all software development costs [1]. Cost and time can be significantly reduced by automating software testing processes associated with software development. Numerous software testing issues demand solutions, such as requiring the smallest possible collection of test data while meeting the highest possible coverage standards. Search-based software engineering is recasting software engineering works as an optimization problem.

The regression testing process is re-testing a program after its modification to ensure that its existing, modified, and new components behave correctly. In regression testing, test suite augmentation is used to determine which program code elements are affected

by changes and to develop test cases targeting those parts of the code. Our previous work [2] detected changes and their affected parts in modified code. In this work, we generate test cases and prioritize the test suite to reduce the testing efforts. We proposed a kernel-based fuzzy c-means clustering (KFCM) technique for clustering the generated test cases into untested and unimportant ones. In addition, the grey wolf optimizer (GWO) method is evaluated for test case prioritization (TCP) using local and global search properties. The optimization is achieved by considering the affected elements in some order while generating test cases. Existing and newly developed test cases are combined, and the test case generation algorithms are utilized. Furthermore, the experiment validates that the test case generation algorithm influences the test case optimization, affecting cost and effectiveness.

The TCP techniques exclusively emphasize test cases that are more significant than others. This pro-

---

\*Corresponding author. Babita Pathik, Department of CSE, TIT College, Bhopal. E-mail: babitapathik@gmail.com.

cess helps to develop effective change-testing for the modified version of the software code. Selecting test cases restricts to cover the modified areas of the code. The technique of reducing and selecting the test cases decreases the test suite by eliminating unessential test cases or by picking only significant test cases. The major application area of the work is software evolution in regression testing and very useful for IT industries.

Several naturally inspired algorithms have been applied to priority test cases, such as genetic algorithms [3], particles swarm optimizations [4, 5], and ant-colony optimization [6], etc. In this document, we used a new nature-inspired optimization method, the GWO. The suggested algorithm is explored and evaluated in sample studies using the popular evaluation measure of the Average Percentage of Fault Detection (APFD). This research aims to find a method that prioritizes new test suites to find faults early and increase the fault detection rate.

Mirjalili et al. (2014) presented the GWO as a population-based swarm intelligence algorithm [7]. The researchers have highly considered this algorithm over the past 3–4 years. In this paper, the GWO algorithm is applied variably to extracted features of abstract syntax trees to optimize test suite augmentation. In addition, the algorithm is used to design change testing techniques that find change-related faults and reduce testing effort.

## 2. Related work

TCP is impactful work for regression testing. As the software matures, test suites tend to expand in size, eventually necessitating the use of expensive resources. The primary goal of running regression tests is to ensure that any changes made to a software module do not affect the unmodified components. In this situation, test cases created for revised software may produce correct results for that version alone. These test cases may produce erroneous outcomes for future software versions generated due to modifications to the first version. Regression testing may be useful in this case.

Simulated Annealing and Genetic Algorithm (GA) is applied for prioritizing test cases [8, 10]. The GA-based technique is used for TCP. In regression testing, TCP prioritizes test cases using the information available.

There are some metaheuristic search-based techniques used for TCP. To give priority to test cases

Firefly algorithm is used [11]. The firefly algorithm is used with a fitness function that was determined by a model called similarity distance to optimize the ordering of the test cases. Rustam and Aini [12] applied fuzzy kernal c-means algorithm for intrusion detection.

Cuckoo Search (CS) algorithm is suggested for the selection and prioritization of test cases. [13]. A CS method and Ant Colony Optimization (ACO) algorithm with modification opted for optimising the test cases in less time [14]. A Bat inspired algorithm is used the bat's pulse emission rates for priority test cases with loudness and echolocation [15, 16].

For regression testing, all important change propagation paths are identified, and the method for handling several more complicated changes is expanded [17]. Gupta, N et al. (2018) presented a detailed study on the generation of test cases for regression for web-based applications [18]. Create two more Adaptive Test Prioritization strategies based on the fuzzy analytical hierarchy process and the weighted sum model [19]. A fuzzy expert system used for TCP in [20]. ACO based algorithm is used to optimize regression test [21]. A local beam search base method is used for TCP [22]. The Cuckoo Search and Bee Colony Algorithm method is used for test case optimization and path convergence generation in a short time [23].

We conclude from the above literature that these algorithms have ended with higher time complexity issues in optimizing and prioritizing test cases. So, the major challenge was finding a method that reduced the time by tracing the faults on test cases as early as possible.

## 3. Methodology suggested

The regression testing method includes ensuring that any system updates carry out successfully and fulfil the expected expectations. Regression testing entails testing a variety of characteristics to ensure that they have not been damaged by changes to a specific feature or function. The ordering of test cases is being carried out to increase the efficacy of regression testing, and the regression testing test case priority problem is involved.

Strong research has been carried out on increasing the accuracy of regression tests, e.g., (1) identifying test cases that do not need to rework on a modified software version, (2) in a test suite that eliminates redundant test cases based on predetermined criteria.

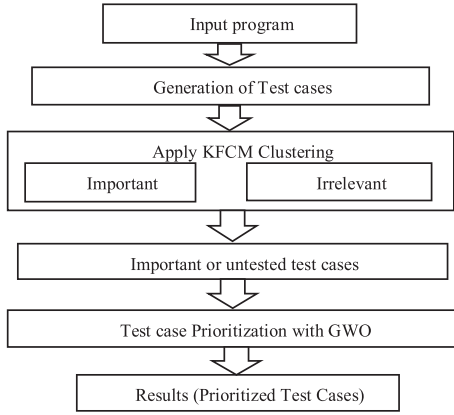


Fig. 1. Flow for proposed test case generation and prioritization.

Therefore, it is necessary to determine whether the current test suites are appropriate for the changes in a program to regulate this efficiency. This paper focuses on the generation of test cases caused by changes. The changes either affect the software behaviour or introduce bugs.

Our new technique derives test cases from the considered applications. The suggested methodology combines various techniques to find the optimized test case sequence. First, a modified KFCM is used for test case clustering. This clustering technique put the generated test cases into two groups: important and unrelated test cases. The cluster of important test cases that are untested may be further processed. TCP is applied on the first cluster using a GWO. The purpose of test case prioritizing is to rank test cases to identify which cases are most likely to find code errors before it is released. Figure 1 shows the flow of our proposed technique.

### 3.1. Test case augmentation

Due to program  $P_0$  and its changed version  $P_n$ , the regression test suite can expose behavioral variations between the two programs and therefore assist developers in finding effects caused by changes or unintentional impacts of modifications introduced in  $P_n$ . Let the  $T_0$  be the test suite for  $P_0$ . It is needed to validate  $P_n$  occurs in regression testing. The test cases on  $T_0$  is usually a concern of testers and various researchers that is more cost-effective with re-utilization in regression test.

Test suite augmentation strategies do not include the reuse of  $T_0$ . Instead, the responsibilities concerned are (1) identifying the changed components

as parts of  $P_n$  for which new test cases are required and (2) developing the test cases which exercise these components. In our previous work [4], we achieved the first requirement and identified the changed code. Here, our focus is on the task of test case generation for identified changed elements on  $P_n$ .

#### 3.1.1. Changed code testing

One reason regression tests exist is to ensure that any alterations made to the software don't negatively affect the functionality of all software modules. When working on regression testing, the best method is to follow TCP approaches that move high-risk test cases to the front of the list and work down from there. A test case clustering method is used to categorize the developed test cases into important and unimportant test cases. A KFCM method is used for clustering purposes. TCP starts with sorting test cases from clustered relevant cases. A TCP's primary purpose is to find a test case order that will help to maximize the possibility of finding software defects earlier

### 3.2. Clustering with KFCM

With the help of kernel-based FCM techniques, we will prioritize our regression test cases in this research paper. The technique under consideration employs various kernel fuzzy c methods for grouping individual jobs. Our approach is similar to fuzzy c means, which will identify similarities between test cases by calculating metrics on their coverage. A method using kernel functions in place of Euclidean distance to strengthening linear machine capabilities. Samples are mapped to high dimensional space, increasing the disparities between the cluster centers and eliminating FCM shortcomings.

Our system uses an algorithm called clustering for grouping current test cases based on the similarity of coverage measures. The following Equation (1) summarizes the objective function (OF) of the proposed kernel based fuzzy c-means method.

$$J_r(U, S, A) = \sum_{i=1}^n \sum_{j=1}^c (\mu_{i,j})^r (x_i - V_j) A (x_i - V_j) \quad (1)$$

Where  $1 \leq i \leq n$ ,  $1 \leq j \leq c$  and  $0 \leq \mu_{i,j} \leq 1$

$U = (\mu_{i,j})_{n \times c}$  = membership function of  $i$ th element of  $j$ th cluster

$S$  = set of data points =  $\{x_1, x_2, \dots, x_n\} \subseteq \mathbb{R}^m$  ( $\mathbb{R}$  = realnumber,  $m$  = dimensions)

$A$  = Positive definite matrix  
 $n$  = no of elements in a set  
 $c$  = no of clusters  
 $r$  = degree of fuzziness for partition  
 $x_i$  =  $i$ th element of a set

$$\sum_{i=1}^n \mu_{i,j} = 1, i = 1, 2, \dots, n \quad (2)$$

$$\sum_{i=1}^c \mu_{i,j} > 0, j = 1, 2, \dots, c \quad (3)$$

$V_j$  = mean of  $i$ th point on  $j$ th cluster and calculated as Equation (4):

$$V_j = \frac{\sum_{i=1}^n x_i (\mu_{i,j})^r}{\sum_{i=1}^n (\mu_{i,j})^r}, j = 1, 2, \dots, c \quad (4)$$

The goal is to minimize OF with Equation (5):

$$\text{Optimal } OF = \min \sum_{i=1}^c \sum_{x \in A_i} \|x_i - V_i\|^2 (\mu_{i,j})^r \quad (5)$$

$$V_i = \text{mean of } A_i$$

### 3.2.1. KFCM

The KFCM is obtained by combining FCM with the kernel function, and the optimal objective function is obtained as Equation (6):

$$J_r(U, S, A) = \min \sum \sum (\mu_{i,j})^r k(x_i, x_i) - 2k(x_i, V_j) + k(V_j, V_j) \quad (6)$$

Define kernel function as  $k(x, y) = \langle f(x), f(y) \rangle$  where  $\langle f(x), f(y) \rangle$  is an inner product [28].

$$X = \{x_1, x_2, \dots, x_n\} \subset R^d, x_i \in R^d.$$

Where  $f: R^d \rightarrow F$ ,  $R^d$  = data space,  $F$  = feature space,  $x, y$  are objects in  $R^d$ . Distance between  $f(x)$  and  $f(y)$ , by using kernel function, defined as Equation (7) [28]:

$$\begin{aligned} d^2(f(x), f(y)) &= \|f(x) - f(y)\|^2 \\ &= f(x)^t f(x) - 2f(x)^t f(y) + f(y)^t f(y) \quad (7) \\ &= k(x, x) - 2k(x, y) + k(y, y) \end{aligned}$$

We have considered these kernels for our work. The below steps are followed for clustering through KFCM:

### Process for KFCM:

Step 1: Initialization of the number of test cases  $\tau$ , number of clusters  $c$  ( $c \geq 2$ ), and random value  $r$ :

Step 2: Start with fuzzy  $c$  partition (FCP) of  $U$  and  $S$

Step 3: Calculate the mean (cluster center) using Equation (8) as:

$$\begin{aligned} V_j &= \frac{\sum_{i=1}^n \tau_i (\mu_{i,j})^r}{\sum_{i=1}^n (\mu_{i,j})^r} \\ \Rightarrow V_j &= \frac{\sum_{i=1}^n k(x, y) (\mu_{i,j})^r}{\sum_{i=1}^n (\mu_{i,j})^r} \quad (8) \end{aligned}$$

In KFCM  $\tau_i$  represented by kernel function  $k(x, y)$ .

Step 4: Calculate the membership function of  $i$ th data of  $j$ th cluster as Equation (9):

$$\mu_{i,j} = \frac{1}{\sum_{k=1}^c \left[ \frac{(d_{i,j})^2}{(d_{i,k})^2} \right]^{\frac{1}{r-1}}} r \geq 1 \quad (9)$$

Where  $(d_{i,j})^2 = k(x_i, x_i) - 2k(x_i, V_j) + k(V_j, V_j)$

Step 5: Go to step 3 until convergence criteria are not satisfied. (The data point belongs to a particular cluster or not)

We will predict an error using this kernel-based FCM. The final phase is error localization, in which we will determine the position and cause of an issue. The clustering process is used to categorize the developed test cases as either important or unimportant. Next to test case clustering, important/untested test cases are examined for test case priority. The purpose of prioritizing test cases is to identify the sequence of test cases that optimizes the chance of discovering source code defects early.

## 3.3. G for TCP

### 3.3.1. TCP

The test case selection for change code testing is an important part of our study. PT represents all potential orderings of  $T$ , and function  $f$  takes a given ordering of  $T$  and generates a corresponding specific value. It's

defined below:

Given: test suite  $T$  has several permutations PT, and the function  $f$  takes PT and turns it into real numbers.

Find: a  $T'$  that satisfies the constraints  $(\forall T)(T \neq T')$ .

There are several possible priorities mentioned here:

- Tting team might desire to enhance a fault detection rate for the test suite or the probability of discovering defects sooner in executing the test suite for regression tests.
- Tm may desire to improve the rate at which a test suite detects high-risk errors, therefore discovering them sooner during the testing process.
- The team may desire to enhance the probability of discovering errors associated with certain code modifications sooner in the regression testing process.

The priority levels have several conceivable aims. A grey wolf pack's social order is replicated by the GWO algorithm.

### 3.2. GWO

GWO is a population-based swarm intelligence system motivated by the grey wolf dominance hierarchy [10]. Grey wolves coexist with humans. Each pack comprises between 5 and 12 people. Assume that each wolf is scouring the search space for a solution. The variable  $w_i = w_{i1}, w_{i2}, \dots, w_{in}$  denotes the position vectors of the wolves in the search space, whereas the size of the dimension is denoted by  $n$ . There are four grades of members in the pack's dominating hierarchy.

**Social hierarchy:** The group's leader is referred to as the  $\alpha$ , and he or she might be male or female. The  $\alpha$  is the pack's highest-ranking member responsible for hunting, picking a resting spot, and determining when to walk. The  $\beta$  wolves are the second kind of wolf in the pack. They assist the  $\alpha$  in making choices.  $\delta$  is the third level in the dominating hierarchy, and it is subordinate to  $\alpha$  and  $\beta$  members. The pack's weakest members are referred to as  $\omega$ , and they surrender to the members on the preceding top tiers.

**Encircling prey:** Following is the mathematical model for the process of encircling prey given as Equation (10) and (11),

$$\vec{X}(t+1) = \vec{X}_p(t) - \vec{A} \cdot \vec{D} \quad (10)$$

Where

$$\vec{D} = \left| \vec{C} \cdot \vec{X}_p(t) - \vec{A} \cdot \vec{X}(t) \right| \quad (11)$$

Coefficient vectors Where  $\vec{A}$  and  $\vec{C}$  are,  $t$  is the current iteration,  $\vec{X}_p$  is the position vector of prey and  $\vec{X}$  denote the position vector of the grey wolf.

The vectors  $\vec{A}$  and  $\vec{C}$  are calculated as follows:

$$\vec{A} = 2 \cdot \vec{a} \cdot \vec{r}_1 \cdot \vec{a} \quad (12)$$

$$\vec{C} = 2 \cdot \vec{r}_2 \quad (13)$$

where elements  $\vec{a}$  are decreased linearly from 2 to 0 over the iterations whereas  $\vec{r}_1, \vec{r}_2$  are the random vectors in  $[0; 1]$ . The fitness function (as defined in the issue specification) is used to estimate the wolves' location.

**Hunting procedure:** The best wolves are categorized as the first solution, denoted by  $\alpha$ , the second solution, denoted by  $\beta$ , and the third solution, denoted by  $\delta$ , respectively. Throughout the process of finding the optimal solution, the wolves upgrade their position in accordance with the positions of  $\alpha$ ,  $\beta$ , and  $\delta$ . The initialization step generates the wolf population and initializes the position of each wolf. After initializing the coefficients, the fitness value of each wolf (search agent) is calculated. Following that, the best fitness solutions are chosen as the first, second, and third positions, respectively.

$$\begin{aligned} \vec{X}_1 &= \vec{X}_\alpha - \vec{A}_1 \cdot (\vec{D}_\alpha) \\ \vec{X}_2 &= \vec{X}_\beta - \vec{A}_2 \cdot (\vec{D}_\beta) \end{aligned} \quad (14)$$

$$\begin{aligned} \vec{X}_3 &= \vec{X}_\delta - \vec{A}_3 \cdot (\vec{D}_\delta) \\ \vec{X}(t+1) &= \frac{\vec{X}_1 + \vec{X}_2 + \vec{X}_3}{3} \end{aligned} \quad (15)$$

Where value of  $\vec{D}_\alpha = C_1 \cdot \vec{X}_\alpha - \vec{X}$ ,  $\vec{D}_\beta = C_1 \cdot \vec{X}_\beta - \vec{X}$  and  $\vec{D}_\delta = C_1 \cdot \vec{X}_\delta - \vec{X}$ : are distance vectors. The values of  $\vec{X}_1, \vec{X}_2$  and  $\vec{X}_3$ , are the initial 3 solutions in the population.

The value of the wolves' fitness function is computed using the new locations, and  $\alpha, \beta$ , and  $\delta$  will be picked. The population  $n$  of the grey wolf is the number of test cases obtained from the path coverage on changed or affected components. The GWO method is used to determine the efficiency and usage of each hosted re. The termination of an algorithm

happens after  $t$  termination criteria are met. After the GWO md computes a score, the prioritizing process bins, occurs based othe score generated. zed tt suite is more useful afulfilling the prioritizing objective for modified code in a specific way and is more suited to achieve success in a single release than test suites developed from broad TCP. Last, we discuss how to prioritize regression test cases and reduce testing effort; this methodology is not limited tst-development testing.

#### 4. Implementation details

The proposed method using KFCM with GWO is implemented on Windows 10 with 8GB RAM, Intel Core i7,4690T central processing unit, 2.50 GHz, 64-bit OS, x64 based processor. An anaconda environment is used for the GWO algorithm. We have taken a Python program of the triangle and angle classification problem for empirical evaluation. The generated test cases are clustered and considered for prioritization for regression testing. To quantify the aim of enhancing the rate of fault detection for a portion of the test suite, we utilize a measure called APFD.

##### 4.1. Prioritized test suite sequence using GWO

We have used the GWO algorithm to prioritize test case sequences in the test suite for early fault detection for regression testing. The total test case 5 of test suite  $T$  is initialized as the pack size of wolves. The total population is  $t_1, t_2, t_3, t_4, t_5$ . The position vector  $\vec{X}$  is initialized with test data of dimension  $d = 3$ . We have initialized the vectors  $A$  and  $C$  with some random values. After the first iteration, these values are updated with Equations (12) and (13). After applying the first iteration, we got the three best solutions as test suites  $t_1, t_2, t_3$ . We have chosen a maximum of 50 iterations. Then update the fitness value and update the search agent by Equation (14) and (15). The value of  $\vec{a}$  is updated as Equation (16):

$$\text{Update } \vec{a} \rightarrow 2 - \frac{t}{t_{\max t}} \quad (16)$$

Where  $t$  = current iteration

$\max t$  = maximum no. of iterations

The values of search agents are updated by comparing the values with the fitness value. The algorithm is terminated after the termination criteria are satisfied. After termination, we have achieved a prioritized test

Table 1  
Tests and faults matrix

| Test Cases | Faults Findings |       |       |       |       |       |
|------------|-----------------|-------|-------|-------|-------|-------|
|            | $f_1$           | $f_2$ | $f_3$ | $f_4$ | $f_5$ | $f_6$ |
| $t_1$      |                 |       | ✓     | ✓     |       | ✓     |
| $t_2$      |                 | ✓     |       | ✓     | ✓     |       |
| $t_3$      |                 | ✓     | ✓     |       | ✓     | ✓     |
| $t_4$      |                 | ✓     | ✓     |       |       | ✓     |
| $t_5$      | ✓               | ✓     | ✓     | ✓     | ✓     | ✓     |

case sequence which helps to detect maximum faults as early as possible. The prioritized test sequence  $t_3 - t_4 - t_2 - t_5 - t_1$  is generated.

##### 4.2. Average percentage of fault detection

This parameter indicates the rate of fault detection with the percentage of test suite execution. It is a metric used to analyze and compare the performance of different approaches for priority test cases. To measure how quickly a prioritized test suite discovers problems during test suite performance, we utilize a weighted average number of faults discovered or APFD. These values vary between 0 and 100; higher APFD figures indicate faster (better) error detection rates. The APFD is derived by averaging the proportion of defects identified during the test suite's execution. The following formula, as in Equation (17), may be used to determine the APFD:

$$APFD = 1 - \frac{(Tf_1 + Tf_2 + \dots + Tf_m)}{mn} + \frac{1}{2n} \quad (17)$$

Where:

$Tf_m$  test suite contains  $n$  number of test cases  $f$  set of  $m$  faults.

$Tf_1$  is the first test case in  $T$  that exposes faults  $i$ .

The number of test cases or test paths  $n = 5$  for the example we have considered. So  $T = \{t_1, t_2, t_3, t_4, t_5\}$  that detects  $m = 6$  number of faults  $F = \{f_1, f_2, f_3, f_4, f_5, f_6\}$  on the basis of the branch coverage prioritization technique (Rothermel et al., 2001).

#### 5. Results and discussion

Table 1 consists of the test cases and their corresponding detected faults in the form of a matrix.

Test case  $t_1$  detects 3 faults from a total of 6 faults which is 50% of fault detected when 0.2 test case of test suite  $T_1$  is used. The next test case is  $t_2$  have detected 3 out of 6 faults, which means again, 50 % of



Fig. 2. APFD for Non-prioritized test suite.

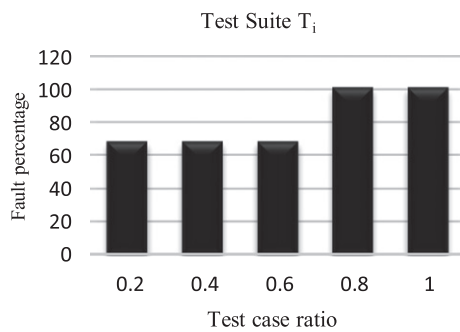


Fig. 3. APFD for Prioritized test suite.

faults have been detected by 0.4 test cases of the same test suite. For  $t_3$ , the total fault detected is 4 out of 6, which is 67%, and 0.6 test cases of the test suite. In the same way, this test suite which has a default sequence  $t_1 - t_2 - t_3 - t_4 - t_5$  gives an APFD of around 56%.

The prioritized test suite with a test sequence obtained from GWO algorithm and the order is changed from  $T_1$  to  $T_i$  at  $i$ th iteration as  $t_3 - t_4 - t_2 - t_5 - t_1$ . This prioritized order gives APFD 91%. The value of APFD for  $T_i$  is much higher than the APFD for  $T_1$ . Higher APFD values imply faster rates of error detection. Figure 2 shows a comparative chart for APFD for a non-prioritized test suite. Figure 3 depicts a comparative chart for APFD for the prioritized test suite. It is evident that this order leads to the quickest identification of the most defects and shows an optimum APFD order of 91.

The time obtained for generating a prioritized test case on a different number of iterations is given in Table 2.

The time taken for various iterations gradually increases accordingly.

As shown in Table 2, the 10th iteration takes 334.9ms, and the 100th takes 916.83 ms of time, respectively.

Table 2

Time elapsed to prioritized test cases on the number of iterations

| Number of iterations | Time elapsed (ms) |
|----------------------|-------------------|
| 10                   | 334.9             |
| 30                   | 516.6             |
| 50                   | 534.5             |
| 70                   | 728.26            |
| 100                  | 916.83            |

Table 3

Comparative analysis

| Technique           | Target code   | APFD       |
|---------------------|---------------|------------|
| Cuckoo search       | Java          | 70.16%     |
| CSBCA               | Java          | 65%        |
| <b>Proposed GWO</b> | <b>Python</b> | <b>91%</b> |

The significance of results drawn from the proposed method is interpreted as the proper ordering of test cases in the test suite finding maximum faults in a few iterations and in less time.

### 5.1. Comparative analysis

To evaluate the suggested method's performance, we compare its outcome with current methods. The result of our technique is compared with other prioritization techniques based on APFD in a generalized way.

The proposed technique gives better results for APFD in comparison to other methods. Panwar D et al. (2018) proposed a cuckoo search-based method that ended with a 70.17% fault detection rate [14], and CSBCA based approach of Lakshminarayana P (2021) achieved a 65% fault case detection rate [23]. Table 3 shows the various technique suggested and their fault detection rate.

Table 3 shows that our projected framework based on the GWO algorithm is achieving a better fault detection rate for TCP than other techniques mentioned. This increases the calculation efficiency and demonstrates that our proposed technique is more efficient than the existing methods.

## 6. Conclusions

The test case generating approaches are designed to create as many test cases as possible for each scenario. Then, suggested a framework using GWO based algorithm to prioritize the test cases. Our suggested technique was assessed using the APFD assessment measures. The time required for priori-

tizing test cases is significantly reduced compared to previous approaches, and the results are also accurate. The results demonstrate that the GWO beats the other classification strategy by enabling very high accuracy. As a result, we can see that our suggested approach outperforms previous work in terms of regression test case prioritizing.

### Conflict of interest

Authors have no conflict of interest.

### References

- [1] Glenford J. Myers, Art of Software Testing, John Wiley & Sons, Inc., USA, 1979.
- [2] B. Pathik and M. Sharma, Source code change analysis with deep learning based programming model, *Automated Software Engineering* **29** (2022), 1–25.
- [3] Arvinder Kaur, and Shubhra Goyal, A genetic algorithm for fault based regression test case prioritization, *International Journal of Computer Applications* **32** (2011), 975–8887.
- [4] Abraham Kiran Joseph, G. Radhamani and V. Kallimani, Improving test efficiency through multiple criteria coverage based test case prioritization using Modified heuristic algorithm, in *Proceedings of International Conference on Computer and Information Sciences*, IEEE, 2016, pp. 430–435.
- [5] Muhammad Khatibsyarhini, Mohd Adham Isa and D.N. Abang Jawawi, A Hybrid Weight-Based and String Distances Using Particle Swarm Optimization for Prioritizing Test Cases, *Journal of Theoretical & Applied Information Technology* **95**(12) (2017).
- [6] Dongdong Gao, Xiangying Guo and Lei Zhao, Test case prioritization for regression testing based on ant colony optimization, in *Proceedings of International conference on software engineering and service science*, IEEE, 2015, pp. 275–279.
- [7] Seyedali Mirjalili, Seyed Mohammad Mirjalili and Andrew Lewis, Grey wolf optimizer, *Advances in Engineering Software* **69** (2014), 46–61.
- [8] R. Uma Maheswari and D. Jeya Mala, Combined genetic and simulated annealing approach for test case prioritization, *Indian Journal of Science and Technology* **8** (2015), 1.
- [9] Yiling Lou, Dan Hao and Lu Zhang, Mutation-based test-case prioritization in software evolution, in *International Symposium on Software Reliability Engineering*, IEEE, 2015, pp. 46–57.
- [10] Fang Yuan, Yi Bian, Zheng Li and Ruilian Zhao, Epistatic genetic algorithm for test case prioritization, in *International symposium on search based software engineering*, Springer, Cham, 2015, pp. 109–124.
- [11] Muhammad Khatibsyarhini, et al., Test case prioritization using firefly algorithm for software testing, *IEEE Access* **7** (2019), 132360–132373.
- [12] Zuherman Rustam and Aini Suri Talita, Fuzzy kernel c-means algorithm for intrusion detection systems, *Journal of Theoretical and Applied Information Technology* **81** (2015), 161–165.
- [13] Reetika Nagar, Arvind Kumar, Gaurav Pratap Singh and Sachin Kumar, Test case selection and prioritization using cuckoos search algorithm, in *Proceedings of International conference on futuristic trends on computational analysis and knowledge management*, IEEE, 2015, pp. 283–288.
- [14] Deepak Panwar, Pradeep Tomar and Vijandra Singh, Hybridization of Cuckoo-ACO algorithm for test case prioritization, *Journal of Statistics and Management Systems* **21** (2018), 539–546.
- [15] Anu Bajaj and Om P. Sangwan, Test case prioritization using bat algorithm, *Recent Advances in Computer Science and Communications (Formerly: Recent Patents on Computer Science)* **14** (2021), 593–598.
- [16] Muhammed Maruf Öztürk, A bat-inspired algorithm for prioritizing test cases, *Vietnam Journal of Computer Science* **5** (2018), 45–57.
- [17] R. Santelices, Pavan K. Chittimalli, T. Apiwattanapong, A. Orso and M. J. Harrold, Test-suite augmentation for evolving software, in *Proceedings of ACM International Conference on Automated Software Engineering IEEE*, 2008, pp. 218–227.
- [18] Nishant Gupta, Vibhash Yadav and Mayank Singh, Automated regression test case generation for web application: A survey, *ACM Computing Surveys* **51** (2018), 1–25.
- [19] Amanda Schwartz and Hyunsook Do, Cost-effective regression testing through Adaptive Test Prioritization strategies, *Journal of Systems and Software* **115** (2016), 61–81.
- [20] C. Hettiarachchi, H. Do and B. Choi, Risk-based test case prioritization using a fuzzy expert system, *Information and Software Technology* **69** (2016), 1–15.
- [21] A. Ansari, A. Khan, A. Khan and K. Mukadam, Optimized regression test using test case prioritization, *Procedia Computer Science* **79** (2016), 152–160.
- [22] B. Jiang and W.K. Chan, Input-based adaptive randomized test case prioritization: A local beam search approach, *Journal of Systems and Software* **105** (2015), 91–106.
- [23] P. Lakshminarayana and T.V. Suresh Kumar, Automatic generation and optimization of test case using hybrid cuckoo search and bee colony algorithm. *Journal of Intelligent Systems* **30**(1) (2021), 59–72.